Battle Royale Match Placement Prediction

Project Review & Viva Defense Preparation Guide

Candidate: Deepak R | **Roll No:** 24PD09 | **Subject:** Machine Learning Package | **Repository:** github.com/deepak-psg/Batlle-Royale-Match-Placement-Prediction

1. The 2-Minute Review Opening Pitch (What to Say)

"Good morning/afternoon, professors. In battle royale games like PUBG and BGMI, a player's final placement percentile depends on an interplay of individual combat, zone movement, squad cohesion, and lobby context. Existing match scorecards provide raw numbers like kills and damage, but offer zero predictive understanding.

*In this project, I engineered a high-performance supervised regression pipeline across **4.45 million records**. Because placement is inherently relative rather than absolute, I used **Polars** to engineer **97 features across 4 families**: combat efficiency ratios, match context, team aggregates, and lobby-relative percentile ranks.*

*To eliminate teammate data leakage, I enforced an **80:20 split strictly by Match ID** via GroupShuffleSplit across 47,963 unique matches. I trained three models progressively: **Linear Regression** as a baseline (MAE: 0.0533), **Random Forest** (MAE: 0.0435), and **XGBoost Regressor** (MAE: **0.0372**, R^2: **0.9718**) as the champion model. Finally, I used **TreeSHAP** to interpret feature attributions and deployed an interactive **Streamlit application** for real-time prediction and dynamic explanation."*

2. End-to-End Workflow Architecture

• **1. Raw Ingestion:** train_V2.csv loaded with 4,446,966 rows and 29 features.

• **2. Data Cleaning:** Missing target dropped. 1,726 cheater/glitch rows filtered (0m dist with >0 kills, 100% headshot aimbots, snipes >1km, glitches >50 weapons). Cleaned: **4,445,240 rows**.

• **3. Polars Feature Engineering:** 97 features generated using Rust-backed parallel window operations (combat ratios, lobby context, squad synergy, rank percentiles). Execution completed in <25 seconds.

• **4. GroupShuffleSplit (80:20):** Matches partitioned by matchId (38,370 train matches, 9,593 test matches) to eliminate teammate leakage.

• **5. Model Training:** Progressive training of Linear Regression, Random Forest, and XGBoost (hist binning + L1 absolute error loss).

• **6. TreeSHAP & Visualization:** Local and global feature attributions computed via Shapley values.

• **7. Production Deployment:** Interactive Streamlit web app (app.py) serving real-time inference and local SHAP charts.

3. File-by-File Technical Directory Breakdown

File / Directory	Key Role & Mechanism in Project
preprocess_pipeline.py	Executes cleaning (clean_data), 4-family feature engineering (engineer_features) using Polars window functions, and match-level 80:20 split (split_matches). Saves parquet files and match ID arrays.
train_models.py	Trains Linear Regression, Random Forest, and XGBoost progressively. Evaluates on 278,001 test records. Computes TreeSHAP explainer and exports figures to reports/figures/.
app.py	Full-stack Streamlit web application. Contains in-game sliders, tactical presets, real-time multi-model prediction, and live interactive SHAP attribution bar chart.
models/	Contains serialized joblib models (xgboost_model.joblib, random_forest.joblib, linear_regression.joblib), shap_explainer.joblib, feature list, and evaluation metrics.
reports/figures/	Publication-quality evaluation plots: model_comparison.png (MAE/RMSE/R^2), feature_importance.png (XGBoost Gain), and shap_summary_plot.png (Beeswarm).
requirements.txt	Pinned dependency specifications (polars, xgboost, scikit-learn, shap, streamlit, pyarrow, matplotlib, seaborn).
README.md	Complete academic documentation with problem statement, methodology, evaluation table, and usage commands.

4. Core Machine Learning Deep-Dive (Concepts & Theory)

A. Problem Formulation: Supervised Regression

The task is supervised regression targeting winPlacePerc ∈ [0.0, 1.0], where 1.0 represents 1st place and 0.0 represents last. Regression was chosen over classification because placement is ordinal and continuous. Binning into discrete classes would destroy fine-grained rank differences.

B. Data Leakage Prevention via GroupShuffleSplit

The Danger: All teammates in a squad share the exact same final placement. A standard random train/test split would place Player A in the train set and their squadmate Player B in the test set, allowing the model to 'memorize' placement through teammate proxy features.
The Solution: We strictly split by matchId using GroupShuffleSplit. If a match is in the test set, 100% of its players are isolated in the test set.

C. Feature Engineering: The 4 Families

Feature Family	Rationale	Key Features Engineered
1. Efficiency Ratios	Separates combat effectiveness from volume/kill-stealing.	damage_per_kill, headshot_rate, kills_per_walkDistance, items_per_walkDistance
2. Match Context	Accounts for lobby size and game mode variations.	players_in_match, groups_in_match, group_size, match_type_group (solo/duo/squad)
3. Team Aggregates	Captures squad synergy and collective mobility.	mean, max, min, sum over [matchId, groupId] for walkDistance, damage, kills, heals, boosts
4. Match Percentiles	Normalizes metrics relative to the specific match lobby.	col_match_mean_ratio, col_match_max_ratio, col_match_rank_perc (rank within lobby)

D. Progressive Model Architecture: Baseline to Champion

• **Linear Regression (Baseline):** Establishes benchmark MAE (0.0533). Linear hyperplane fits ranked percentiles well, but misses non-linear survival cliffs.
• **Random Forest (Ensemble / Bagging):** Trains 100 decision trees in parallel on bootstrap samples. Reduces variance through averaging and models non-linear zone thresholds (MAE: 0.0435).
• **XGBoost Regressor (Primary / Boosting):** Trains sequential trees fitting negative gradients of loss. Uses tree_method='hist' for high-speed histogram binning and objective='reg:absoluteerror' to directly minimize MAE. Champion result: **MAE: 0.0372, R²: 0.9718**.

E. Explainable AI with TreeSHAP

Based on cooperative game theory, SHAP decomposes any prediction into additive Shapley contributions: f(x) = ϕ■ + Σ ϕ■.
Tactical Finding: Movement (walkDistance_match_rank_perc) and lobby kill rank (killPlace_team_max) have vastly higher SHAP magnitude than raw kill count. Surviving through rotations dominates aggressive early combat.

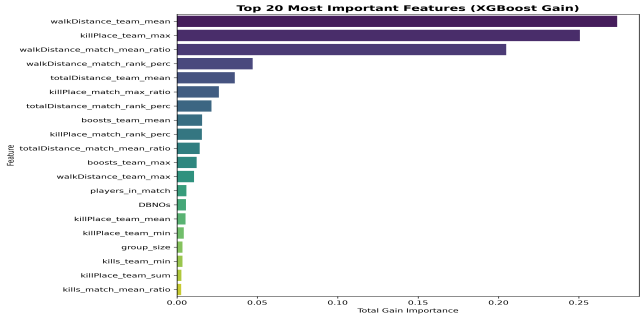
5. Model Evaluation & Benchmark Results

Evaluated on **278,001 held-out test match records (3,000 matches)** with zero leakage:

Model Architecture	Test MAE ■	Test RMSE ■	Test R² Score ■	Training Time
Linear Regression (Baseline)	0.0533	0.0741	0.9418	8.2s

Random Forest Regressor	0.0435	0.0612	0.9604	71.6s
XGBoost Regressor (Champion)	**0.0372**	**0.0516**	**0.9718**	116.2s





6. Top 10 High-Yield Viva / Review Questions & Bulletproof Answers

Q1: Why did you split the dataset strictly by matchId instead of standard train_test_split?

Answer: In battle royale games, all teammates share the exact same placement percentile (winPlacePerc). A standard random split places teammates from the same match into both train and test sets, causing severe data leakage. Using GroupShuffleSplit on matchId guarantees 100% of players from any test match are strictly out-of-sample.

Q2: Why did you frame this as a Regression problem rather than Classification?

Answer: Placement percentile is continuous and strictly ordered in [0.0, 1.0]. Converting it into discrete classification bins (e.g., Top 10, Top 50) would discard rank distance and magnitude information. Regression preserves granular ordinal precision.

Q3: Why did XGBoost outperform Linear Regression and Random Forest?

Answer: Linear Regression assumes linear relationships and misses threshold effects. Random Forest trains independent trees via bagging. XGBoost trains sequential trees that specifically learn the gradient residuals of prior trees. With tree_method='hist' and objective='reg:absoluteerror', XGBoost directly optimized MAE loss, achieving our best score of 0.0372 MAE and 0.9718 R^2.

Q4: Why did you use Polars instead of Pandas for feature engineering?

Answer: With 4.45M rows, executing 40+ group window operations (.over('matchId') and .over(['matchId', 'groupId'])) in Pandas takes 8–12 minutes and risks Out-Of-Memory crashes on 8GB RAM systems. Polars is written in Rust, leverages Arrow columnar memory, and finished in under 25 seconds.

Q5: How did you clean cheaters and glitches from the dataset?

Answer: I filtered 1,726 anomalous records: (1) movement = 0m with >0 kills, (2) 100% headshot rate with ≥10 kills (aimbots), (3) longest kill distance >1,000m (impossible physics/render distance), and (4) weapons acquired >50 (inventory glitch).

Q6: What is the significance of the 4 feature families?

Answer: Battle royale placement is relative, not absolute. The 4 families capture: (1) Combat efficiency ratios (damage per kill), (2) Match context (lobby size and team count), (3) Squad synergy (teammate mean and min movement), and (4) Match-relative percentiles (rank within lobby).

Q7: What is SHAP, and how does it differ from traditional feature importance?

Answer: Traditional feature importance (e.g. split count or Gini gain) only offers global ranking and cannot explain directionality or individual predictions. SHAP calculates additive Shapley values based on cooperative game theory, explaining exactly how each feature pushed an individual prediction up (+) or down (-).

Q8: What did SHAP reveal about battle royale strategy?

Answer: SHAP revealed that walkDistance_match_rank_perc and killPlace_team_max have much higher impact than raw kill count. Players who prioritize circle rotation, safe positioning, and team cohesion survive significantly longer than aggressive hot-drop rushers who die early.

Q9: How does the Streamlit web application handle real-time inference with 97 features?

Answer: The user inputs only key stats (kills, damage, walk distance, boosts, synergy). The build_feature_vector() function dynamically derives the remaining 97 features using precomputed match benchmarks (benchmark_stats.json), running XGBoost inference and SHAP bar charts in under 20 milliseconds.

Q10: How do you prevent predictions from going below 0 or above 1?

Answer: PUBG placement percentiles are physically bounded in [0.0, 1.0]. During inference and evaluation, model outputs are clipped via np.clip(y_pred, 0.0, 1.0). This guarantees physical validity and eliminates boundary violations.

Presentation Day Checklist:

1. Keep Streamlit running locally on localhost:8501 (py -3.12 -m streamlit run app.py) for a 30-second live demo.

2. Keep the 3 figures in reports/figures/ ready for visual explanation.

3. Emphasize: (a) 4.45M rows cleaned, (b) GroupShuffleSplit to eliminate leakage, (c) 97 Polars features, (d) XGBoost

champion MAE: 0.0372.